SkillNavora · Business AI Stack

COMPLETE GUIDE 11 OF 15

RAG

A practical guide with steps, examples and a checklist. Read the chapter, then build the example.

AI FOUNDATIONS · COMPLETE GUIDE

RAG: answers from your own documents

Retrieval-augmented generation (RAG) lets an LLM answer from your documents. Instead of relying on memory, the system finds the most relevant passages first, then asks the model to answer using only those passages, with citations. This guide builds a question-answering assistant over a handbook PDF.

1. The pipeline

Step	Name	What happens
1	Ingest	Load the PDF; extract text; split it into chunks (for example 500 tokens with 50 overlap).
2	Embed and index	Turn each chunk into a vector; store it in a vector index (Supabase pgvector, or any vector database).
3	Retrieve	Embed the question; fetch the top 4 to 6 most similar chunks.
4	Generate	Give the model the question and the chunks; require an answer with citations, or "not in the document".

2. Worked example: ask questions of a handbook

1. Extract the text from the PDF with a library such as `pypdf`. Save each page with its number.
2. Chunk by paragraph, keeping the page number in each chunk's metadata.
3. Create an embeddings table in Supabase with a `vector` column (enable the pgvector extension). Insert each chunk with its embedding.
4. For a question, embed it, run a similarity search (`order by embedding <=> query limit 5`), and build a prompt with the top chunks.
5. Ask the model: answer only from the sources, cite pages like [p. 12], and say "not found" when the sources do not answer.

```
SYSTEM = ("Answer using only the SOURCES. Cite each fact with its page, like [p. 12]. "
          "If the sources do not answer, say: 'Not found in the handbook.'")
prompt = SYSTEM + "\n\nSOURCES:\n" + "\n".join(f"[p. {c.page}] {c.text}" for c in top_chunks) +
"\n\nQUESTION: " + question
```

3. Quality: measure it

- **Retrieval test.** For 20 questions, check that the right page appears in the top 5 results.
- **Answer test.** Check each answer's citation against the page. A confident answer with the wrong citation is a failure.
- **Tune one thing at a time:** chunk size, overlap, number of chunks, then the prompt.

4. RAG versus fine-tuning

Use RAG when the knowledge changes, must be cited, or lives in documents. Fine-tuning changes how the model behaves (style, format), not what it knows. Most business assistants need RAG first.

ACCESS CONTROL

Filter retrieval by the user's permissions. A search that returns a confidential chunk to an unauthorised user is a data leak, even if the answer is never shown.

5. Checklist

- Chunks keep their source and page.
- Retrieval and answer tests on a fixed question set.
- "Not found" behaviour tested.
- Permissions applied at retrieval time.

Reference: the Supabase documentation for pgvector, and the OpenAI or Anthropic API documentation for embeddings and messages.